IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.DOI

Bounded Memory and Bounded Work for Scoped Symbol Tables: A Single-Buffer Design Evaluated on Compiler-Generated Workloads

SHUBHI SINGH[1], ADHYAN JAIN[1], AKSHITH VENKATATARAMANA[1], and ARJUMAN[1]
[1]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India **[AUTHOR: confirm this affiliation for each author and add e-mail addresses]**

Corresponding author: **[AUTHOR: name and e-mail address of the corresponding author]**.

[AUTHOR: state the funding of this work, or that it received no specific funding]

ABSTRACT A compiler that must run inside a fixed memory budget, for example on a microcontroller, needs a symbol table whose size is bounded in advance, whose behavior at the bound is defined, and whose cost per call cannot be driven up by hostile source text. Scoped symbol tables in textbooks and in production compilers rely on a general-purpose allocator and offer none of these properties. This article presents STRATA, a scoped symbol table that keeps its index, its interned names, and an undo stack in one caller-supplied buffer. An operation that does not fit is refused and leaves the table bit-for-bit unchanged. We show that the state of the table after any sequence of operations equals that of a fresh table holding only the live declarations, and we use this property to bound the probability that a limit on index probes refuses a legitimate name. On symbol-table events logged by an instrumented C compiler for eight open-source projects, STRATA needs a fixed buffer 16.4 to 17.9 percent smaller than a conventional single-buffer table, with replay time within 10.7 percent of it and no established difference on most traces, and 23 to 76 percent less heap than its multi-allocation predecessor. Identifiers that collide in the default hash for every seed slow declarations by a factor of 300; a keyed hash and a probe limit remove the slowdown. The guarantees are supported by differential fuzzing, exhaustive enumeration of short operation sequences, and a run on an emulated microcontroller.

INDEX TERMS Algorithmic complexity attacks, compilers, embedded software, hash tables, linear probing, memory management, software testing, symbol tables.

I. INTRODUCTION

A SCOPED symbol table maps identifiers to declarations and forgets the declarations of a block when the block ends [1]. It is one of the oldest data structures in compiler construction [2], and for a compiler on a workstation it is a solved problem: a hash table from the standard library and an undo stack are enough [3]. The situation changes when the compiler itself is the constrained program. A compiler or interpreter front end that runs on a microcontroller, inside a boot loader, or in a tool that must never exceed a fixed allocation has three requirements that the standard solution does not meet.

Bounded memory. The table must live in storage whose size is fixed before the first declaration, without calls to an allocator whose fragmentation and failure behavior are outside the compiler's control.

Defined behavior at the bound. When a declaration does not fit, the table must say so and remain usable. A table that is left half-updated turns an ''out of memory'' diagnostic into an incorrect compilation.

Bounded work on untrusted input. If the source text is supplied by someone else, the cost of one table operation must not depend on that person's choice of identifiers. Hash tables are a known target of algorithmic complexity attacks [4], and seeding a common non-cryptographic hash does not prevent them [5].

The scoped tables we found in textbooks and in production compilers [3], [6], [7] obtain memory from an allocator, do not define their state after a failed allocation, and use unkeyed hashes. They were not designed for these requirements, and for a hosted compiler they need not be [8].

This article asks what it costs to meet all three requirements, and answers by building and measuring. The object of study is STRATA, a scoped symbol table in one buffer of 32-bit words. Its algorithm is the textbook one: one hash index, an undo stack, restoration on scope exit [3]. Our contributions

lie in what is built around that algorithm and in how it is evaluated.

1) **A single-buffer scoped symbol table with an all-or-nothing refusal.** Index, interned names with their innermost declaration, hidden declarations and scope marks share one caller-supplied buffer. An operation that does not fit returns a status code and leaves every word of the state unchanged (Section IV).
2) **A canonical-state property.** After any sequence of operations, the state of the table is exactly the state of a fresh table into which only the still-live declarations were made. Scope exits leave no tombstones and no displaced keys, so the table does not degrade with use, and results about insertion-only linear probing apply to it at all times (Section V).
3) **A bounded mode with an analyzed probe limit.** With a static index and a limit of L probes per call, a name that would need more is refused with nothing changed. We bound the probability of such a refusal for an ideal hash, validate the bound on 240 000 000 simulated insertions, and show that a limit suggested by real traces alone would have been too small (Section VI).
4) **A measured decomposition on compiler-generated workloads.** Against a multi-allocation predecessor we separate what is gained by keeping everything in one allocation-free buffer, which a conventional chained table also achieves, from what the compact design adds. We also report that an apparent time penalty of the compact design, of up to a third on these workloads, was an implementation artifact, and identify its cause (Sections IX and IX-D).
5) **An attack and its defenses, measured.** We construct C identifiers that collide in a MurmurHash3-style hash of packed identifier characters for every seed, measure the resulting slowdown, and measure what a keyed hash and the probe limit cost and achieve (Section IX-G).

We do not claim a new hashing scheme, a faster symbol table, or a result on hardware. The compact table is not faster than the conventional one; the collision construction applies a published technique [5] to a restricted alphabet; and the microcontroller is emulated. Section XI lists what the evidence does not cover.

The rest of the article is organized as follows. Section II reviews related work. Section III states the problem and the threat model. Section IV describes the design, Section V the canonical-state property, and Section VI the bounded mode. Section VII describes how the guarantees were tested. Sections VIII and IX give the experimental setup and results. Sections X and XI discuss the findings and their limitations, and Section XII concludes.

II. RELATED WORK

A. SCOPED SYMBOL TABLES

Graham *et al.* [2] described hashed symbol tables for languages with explicit scope control in 1979. The scheme used here is the one Appel presents for block-structured languages [3]: a single hash table maps each name to its innermost binding, an auxiliary stack records the names bound in each scope, and leaving a scope pops the stack and restores what was hidden. Lookup cost is independent of the nesting depth, in contrast to a stack of per-scope maps [1] that is searched from the innermost scope outward. LLVM's `ScopedHashTable` [6] is a reusable implementation of the same idea, with nodes chained per key and per scope and obtained from an allocator. The Tiny C Compiler [7] keeps a token table and per-token symbol pointers with symbol stacks that are popped at scope exit. All of these allocate per symbol or per node, none defines the state of the table after an allocation fails, and none bounds the work of a call.

Region allocation by object lifetime [9] removes most of the allocator's cost for compilers by releasing a whole region at once. A chained hash table whose entries live in a stack-like arena is the natural single-buffer form of this idea, and we use it as our main baseline (Section VIII-B).

Language runtimes for microcontrollers intern identifiers to save memory. MicroPython, for example, stores each distinct string once in pools of interned strings together with its hash and length [10]. Such pools address the cost of names but not scopes or failure behavior.

B. DELETION IN OPEN ADDRESSING

Removing a key from a linear-probing table normally requires a tombstone or a backward shift [11]. Bender *et al.* [12] showed that the way deletions are implemented has a large effect on the behavior of linear probing over time. Robin Hood hashing [13], which the predecessor of this work used, reorders keys on insertion and therefore also on deletion. A symbol table removes keys in a very particular order: last declared, first removed. We use the observation that in linear probing without displacement the most recently inserted key can be removed by emptying its slot, with no tombstone and no shift (Section IV-C). We do not claim this observation is new; it follows directly from the insertion rule. What we add is its consequence, the canonical-state property of Section V.

That property is related to, but weaker than, history independence [14], [15], where the memory layout must be a function of the stored set alone. In STRATA the layout is a function of the live declarations *and* their order.

C. ANALYSIS OF LINEAR PROBING

For an ideal hash, Knuth gives the expected number of probes of an unsuccessful search in insertion-only linear probing at load α as $\frac{1}{2}\left(1 + 1/(1-\alpha)^2\right)$ [11]. Pagh *et al.* [16] and Thorup [17] analyze linear probing under limited independence by counting the keys that hash into an interval. Our bound in Section VI-B uses the same interval argument with a Chernoff bound for a fully random hash; its only project-specific ingredient is the property that lets an insertion-only argument cover a table with deletions.

D. DICTIONARIES WITH BOUNDED PROBE SEQUENCES

Hash tables that bound the work of a lookup by construction are well known. Cuckoo hashing [18] places every key in one of two positions and so looks up in constant worst-case time; hopscotch hashing [19] keeps every key within a fixed neighborhood of its home slot. Both obtain the bound by *moving* keys on insertion, and both fall back on resizing or rehashing when an insertion cannot be completed. Neither behavior suits the setting studied here. Moving keys destroys the property that the newest key can be removed by emptying its slot, on which scope exit and the canonical-state property rest, and a fixed buffer cannot be resized. The bounded mode of Section VI therefore bounds probes in the simplest possible way, by refusing an insertion that would exceed the bound, and the question becomes how often an honest input is refused. We make no claim that this is a better dictionary in general; it is the variant that composes with last-in-first-out removal and an all-or-nothing refusal.

E. ALGORITHMIC COMPLEXITY ATTACKS

Crosby and Wallach [4] showed that an attacker who chooses the keys can degrade a hash table to a list. Aumasson and Bernstein [5] showed that MurmurHash2 and MurmurHash3 admit multicollisions that hold for every seed, so that a secret seed is no defense, and proposed the keyed function SipHash; the accompanying presentation [20] gives the construction for MurmurHash3. HalfSipHash is the variant with 32-bit words; the Linux kernel documents its one-compression-round, three-finalization-round form as suitable only for mitigating hash-table flooding [21]. Compilers for trusted input commonly choose speed instead: the hash used inside the Rust compiler is deliberately unkeyed because, in its developers' words, such attacks "aren't a concern in the compiler" [8]. We take the opposite case, a compiler that cannot trust its input, and measure both the attack and the defenses on a scoped table.

F. SUMMARY OF DIFFERENCES

Table 1 summarizes the properties relevant here. Entries for other systems state what their public documentation or source code shows; "no" means that we found no such mechanism, not that one could not be added. The differences are properties of implementations rather than of algorithms: every design in the table resolves scopes with the same underlying idea.

III. PROBLEM FORMULATION

A. OPERATIONS AND SEMANTICS

A scoped symbol table supports four operations on a stack of scopes that initially holds one outermost scope.

- `enter`: push a new innermost scope.
- `exit`: remove the innermost scope and all declarations made in it; refused at the outermost scope.
- `declare`(x, v): bind name x to value v in the innermost scope. If x is already bound in that scope the value is replaced; otherwise a new declaration is created, which hides any declaration of x in an enclosing scope.
- `resolve`(x): return the value of the innermost visible declaration of x, or report that there is none.

A name is a nonempty byte string of at most 255 bytes. A value is a 32-bit integer accompanied by an 8-bit type tag; a compiler would store an index into its own declaration records.

We write d for the number of live declarations, $n \leq d$ for the number of distinct names among them, s for the number of open scopes besides the outermost, and $w(x)$ for the number of 32-bit words that name x occupies.

B. REQUIREMENTS

R1 **Fixed storage.** All state resides in one buffer of C words supplied by the caller. No allocator is called.

R2 **All-or-nothing refusal.** Every operation either succeeds or returns a status code and leaves the complete state unchanged.

R3 **Correct scoping.** Every `resolve` returns what the definition above prescribes, given the operations that succeeded.

R4 **Bounded work per call.** There is a constant, independent of d and of the names chosen, that bounds the number of index slots examined by one `declare` or `resolve`.

R5 **Rare refusal of honest input.** Under R4, the probability that a declaration is refused although memory is available must be negligible for names that were not chosen to collide.

R1 to R3 are met by the default mode of STRATA and by the baseline. R4 and R5 are met by the bounded mode, R5 under the assumption stated next.

C. THREAT MODEL

The adversary supplies the source text and thereby every name and the order of all operations. The adversary knows the implementation, including the hash function, but not a 64-bit key that the device draws from a source the adversary cannot predict. The adversary's goal is to make table operations slow or, with a probe limit, to have legitimate declarations refused. We do not consider memory-safety attacks on the surrounding compiler, side channels through which the key might leak, or an adversary who can observe timing with enough precision to recover the key. Exhausting the buffer with many distinct names is always possible and is not an attack on the table: it is the budget doing its job, and R2 keeps the table usable afterwards.

IV. DESIGN OF STRATA

A. LAYOUT

All state lives in one array of 32-bit words (Fig. 1).

The *index* occupies the first m words, m a power of two. A slot is zero when empty; otherwise it holds an 8-bit fingerprint taken from the hash and a 24-bit reference to an identifier record. Collisions are resolved by linear probing, and the index is kept at most three quarters full.

TABLE 1. Properties of Scoped Symbol Tables and Identifier Stores Relevant to This Work.

Design	Scopes with shadowing	One caller-supplied buffer	Defined refusal, state unchanged	No rebuild inside a call	Bound on probes per call	Keyed hash
Undo-stack scheme [3], LLVM `ScopedHashTable` [6]	yes	no	no	no	no	no
Tiny C Compiler symbol stacks [7]	yes	no	no	no	no	no
MicroPython interned strings [10]	no	pools	not stated	n/a	no	no
METIS-X [22] (predecessor)	yes	no	no	no	no	no
Conventional arena table (our baseline)	yes	yes	yes	no	no	no
STRATA, default mode	yes	yes	yes	no	no	optional
STRATA, bounded mode	yes	yes	yes	yes	yes	optional

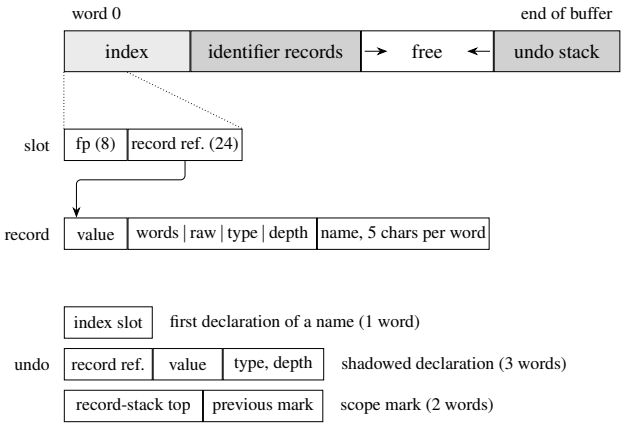


FIGURE 1. Layout of the buffer and of the three kinds of entries. The index and the identifier records grow from the start of the buffer, the undo stack from its end; the free space between them is shared.

The *identifier records* follow the index and grow upward. There is one record per distinct visible name. A record holds the value of the name's *innermost* declaration, a word of metadata (number of name words, an encoding flag, the type tag, and the depth of the scope that made the innermost declaration), and the name. A lookup therefore ends in the record it finds: there is no second step to a declaration record, and its cost does not depend on how often the name is shadowed.

Names consisting only of letters, digits and the underscore are packed five characters to a word, six bits each. The query is packed once and compared word by word, so stored names are never unpacked. Other names are stored as raw bytes.

The *undo stack* grows downward from the end of the buffer. A declaration of a name that was not visible before pushes one word, the index slot of the new record. A declaration that hides an outer one pushes three words: the record reference and the value, type and depth that were overwritten. Entering a scope pushes a two-word mark holding the top of the record stack and the position of the previous mark.

With m index slots, the buffer is used up to

$$M = m + 1 + \sum_{x} \big(2 + w(x)\big) + n + 3\,(d - n) + 2\,s \quad (1)$$

words, where the sum runs over the n distinct visible names and the constant accounts for the reserved reference zero. An ordinary declaration of a new name thus costs $3 + w(x)$ words plus its share of the index. The conventional table of Section VIII-B stores, for every declaration, a chain link, the hash, the value, a length-and-depth word and the unpacked name, which is $4 + \lceil \ell/4 \rceil$ words for a name of ℓ bytes, plus a bucket.

B. DECLARATION AND LOOKUP

Algorithm 1 gives the declaration. It has the form *search, reserve, write*: nothing is written before the last point at which the operation can be refused. A failed search ends at the first empty slot on the name's probe path, which is exactly where linear probing would insert the name; the slot is remembered and reused, so a declaration walks the path once.

A lookup packs and hashes the name and probes from its home slot until it finds a slot whose fingerprint, metadata and name words match, or an empty slot. The fingerprint is a filter only; a match is always confirmed by comparing the name.

C. SCOPE EXIT WITHOUT TOMBSTONES

Leaving a scope pops the undo stack down to the scope's mark, newest entry first. A three-word entry restores the value, type and depth it saved. A one-word entry empties the index slot it names. Then the top of the record stack is reset to the value in the mark, which discards the records created in the scope.

Emptying a slot is sufficient because of the order of removal. In linear probing without displacement, a key is placed in the first empty slot of its probe path and never moves, so every slot between a key's home and its position was occupied, at the time of insertion, by an older key. The key being removed is the newest key in the index. It therefore lies on no other key's path, and emptying its slot cuts no path. The same holds for the next entry, which is the newest of those that remain. No tombstone is written and no key is shifted.

D. ALL-OR-NOTHING REFUSAL

Each operation validates its arguments, reserves everything it needs, and only then writes. In fixed mode the reservation is a comparison of counters; when the index must grow, the growth and the request are checked together before anything moves. After a successful reservation no step can fail. Scope exit cannot fail except at the outermost scope, where it returns before doing anything. Consequently every refusal (no memory, nesting deeper than 16 383, an empty or over-long name, and the probe limit of Section VI) leaves the state exactly as it was. Section VII describes how this was checked.

Algorithm 1 Declaration of name x with value v

Input: name x, value and type v; current depth δ; probe limit L (bounded mode) or $L = \infty$

Output: a status; the state is unchanged unless the status is `ok` or `redeclared`

$k \leftarrow \text{PackAndHash}(x)$ {refuses empty and over-long names}
$(r, e) \leftarrow \text{Find}(k, L)$ {record, or first empty slot within L probes}
if $r \neq$ `nil` **and** $depth(r) = \delta$ **then**
  overwrite value and type of r;
  return `redeclared`
else if $r \neq$ `nil` **then**
  if fewer than 3 free words **then**
    return `no-memory`
  end if
  push $(r, value(r), type(r), depth(r))$ on the undo stack
  overwrite value, type of r; $depth(r) \leftarrow \delta$;
  return `ok`
end if
if fewer than $3 + w(x)$ free words, or the index would exceed its load limit and cannot grow **then**
  return `no-memory`
end if
if $e =$ `nil` **then**
  return `probe-limit`
end if
append a record for x with v and δ; store its reference and fingerprint in slot e
push e on the undo stack;
return `ok`

E. GROWING THE INDEX

In the default mode the index doubles when it would exceed its load limit. The identifier records are then moved up as one block, which their relative references permit, and the index is rebuilt by reinserting the names oldest first, so that the property used in Section IV-C holds again. One-word undo entries, however, name index slots, and those change. Each record needs to find its own undo entry again without a search and without scratch memory, which a fixed buffer does not have. Before the rebuild, each one-word entry and its record therefore exchange one word: the record's value word temporarily holds the position of the entry, and the entry holds the value. During reinsertion each record follows that position, takes its value back, and writes its new slot into the entry. Every name is hashed once.

A table that owns a growable buffer can also extend the buffer. When only the buffer grows, the index and the records do not move and the recorded slots stay valid, so nothing is rehashed. Section IX-D shows what happens when this distinction is missed.

F. COMPLEXITY

Let w be the number of words of the name involved. For an ideal hash at load at most $3/4$, a lookup and a declaration examine an expected constant number of slots [11] and compare w words per fingerprint match. Entering a scope takes constant time. Leaving a scope takes time proportional to the number of declarations made in it. A rebuild takes time proportional to n and, with doubling, constant amortized time per declaration. In the default mode a single call can therefore take time proportional to n; the bounded mode removes this case.

V. THE CANONICAL-STATE PROPERTY

Proposition 1: Consider a table whose index size is fixed. After any sequence of operations, including refused ones, let D be the list of declarations that are still live, in the order in which they were made, together with the scopes that are still open. Then every word of the table's state equals the corresponding word of a fresh table into which exactly the scope entries and declarations of D were made.

Proof: By induction on the number of operations. A refused operation changes nothing. A successful `enter` or a successful declaration of a new or shadowing name appends to D and performs on the table exactly what it would perform on the fresh table for the shorter list, which by the induction hypothesis is in the same state. A redeclaration in the innermost scope overwrites the value of a live declaration in place, in both tables. A scope exit removes the declarations of the innermost scope, a suffix of D, newest first. Undoing the newest declaration restores exactly the words that making it changed: a three-word entry puts back the saved value, type and depth, and a one-word entry empties the one index slot that the insertion filled, since insertion without displacement moves no other key; the record is dropped by resetting the stack top. Words beyond the two stack tops are not part of the state. After the exit the table therefore equals the fresh table for D without that suffix. ■

With an index that grows, the statement holds for everything except the index size, because a rebuild reinserts the names in the order of D.

The proposition has two consequences that the rest of the article uses. First, the table does not age: no sequence of scope entries and exits lengthens probe paths. Second, at every moment the index is exactly the index that insertion-only linear probing would build for the currently visible names. The classical analysis of insertion-only tables [11] therefore applies to STRATA at all times, although the table performs deletions.

VI. BOUNDED MODE

A. STATIC INDEX AND PROBE LIMIT

In the bounded mode the caller fixes the number of index slots m when the table is created. The index is never rebuilt; a declaration that would exceed the load limit is refused like one that exceeds the buffer. The caller may also set a probe limit L. A new name is then inserted only if an empty slot lies

TABLE 2. Smallest Probe Limit for Which the Bound (2) Is Below 10^{-6}.

Maximum load α	ρ	Per declaration	Over 10^6 declarations
0.750	0.9630	455	821
0.500	0.8244	81	153
0.375	0.7006	43	82

among the first L slots of its probe path, and is refused with the status `probe-limit` otherwise (Algorithm 1). Since a stored name never moves and scope exit only empties slots, every stored name lies within L slots of its home, and a search may stop after L slots. One call thus examines at most L index words and, per fingerprint match, compares at most 64 name words: a bound that depends on L and the name-length limit only (requirement R4). A name that hides a visible one needs no slot and is never refused for this reason. Configurations that would silently lack the bound, such as a probe limit without a static index, yield a table that refuses every call.

The bound does not cover scope exit, which is proportional to the number of declarations in the scope being left, nor name packing, which is linear in the name length.

B. HOW OFTEN DOES THE LIMIT REFUSE AN HONEST NAME?

Proposition 2: Let the index have m slots and hold n names whose home slots are independent and uniformly distributed, and let $\alpha = n/m < 1$ and $\rho = \alpha\, e^{1-\alpha}$. The probability that a new name with an independent uniform home slot is refused at probe limit L is at most

$$\frac{\rho^L}{1-\rho}. \tag{2}$$

Proof: Let h be the home slot of the new name. The name is refused only if slots $h, \ldots, h+L-1$ (indices modulo m) are all occupied. Let $u \le h$ be the first slot of the maximal run of occupied slots containing h. Slot $u-1$ is empty. By Proposition 1 the index is an insertion-only table, in which a slot that is empty has always been empty, so no key was ever carried across slot $u-1$. Each of the $k = h+L-u \ge L$ keys stored in slots $u, \ldots, h+L-1$ therefore has its home in that interval. Hence, for some $k \ge L$, at least k of the n names have their home in the fixed interval of length k ending at slot $h+L-1$. The number of names with home in a fixed interval of length k is binomial with mean αk, and by the Chernoff bound the probability that it is at least k is at most $\left(e^{1-\alpha}\alpha\right)^k = \rho^k$. Summing over $k \ge L$ gives (2); $\rho < 1$ because $\alpha e^{1-\alpha}$ is increasing on $(0,1)$ with value 1 at $\alpha = 1$. ∎

Because ρ increases with α, the bound for the load limit of a table covers every lower load, and for a compilation that declares N new names the probability of any refusal is at most $N\rho^L/(1-\rho)$. Table 2 lists the smallest limits that the bound supports. For the default load limit of $3/4$, $\rho = 0.963$ and the bound is useful only for limits of several hundred; a limit of 512 gives at most 1.1×10^{-7} per declaration.

The proposition assumes independent uniform home slots. For names that were not chosen with knowledge of the hash this is the usual modeling assumption; for names chosen by an adversary it holds only if the hash is keyed.

C. KEYED HASH

The default hash is the 32-bit MurmurHash3 mixer [23] applied to the name words. As an option, the table hashes with HalfSipHash with one compression round and three finalization rounds [5], [21] under a 64-bit key supplied by the caller. Our implementation is checked against the reference code [24] (Section VII). We rely on the function's design goal, indistinguishability from a random function for an adversary without the key, and make no cryptographic claim of our own. With a keyed hash, Proposition 2 applies to adversarial input under that assumption; without one, the probe limit still enforces R4 but an adversary can cause refusals.

D. COLLIDING IDENTIFIERS FOR THE DEFAULT HASH

To quantify the threat we construct colliding names. The mixer processes a name word k as $h \leftarrow \mathrm{rotl}_{13}\big(h \oplus f(k)\big)\cdot 5 + c$, where f multiplies, rotates and multiplies and is a bijection on 32-bit words that does not involve the seed. Following Aumasson and Bernstein [5], [20], flipping bit 18 of $f(k_1)$ flips bit 31 of the rotated word; multiplying by 5 and adding c leaves that difference in bit 31 for every h; and flipping bit 31 of $f(k_2)$ in the next word cancels it. Two consecutive words (k_1, k_2) can thus be exchanged for (k_1', k_2') with $f(k_1') = f(k_1) \oplus 2^{18}$ and $f(k_2') = f(k_2) \oplus 2^{31}$ without changing the hash, whatever the seed.

In our setting the words are not free: a word must be a packing of five identifier characters, which requires its two top bits to be zero and none of its five six-bit codes to be zero, and the first character of a name must not be a digit. A random search finds word pairs that satisfy these constraints for both alternatives within a fraction of a second, since about one word in four is a valid packing. Concatenating t such pairs gives 2^t distinct identifiers of $10t$ characters with one common hash. With $t = 15$ this yields 32 768 valid C identifiers of 150 characters. We verify that they collide under the table's built-in seed and under eight further seeds.

VII. VERIFICATION

The guarantees of Sections IV to VI are argued informally. To support them we test at four levels. None of this is a proof.

Unit and randomized differential tests. A test program compares the table with an independent reference model built from the standard library after every operation, in configurations that cover packed and raw names, fixed and growable buffers, a dense and a sparse index, allocators that fail, and a hook that forces every key into one probe cluster. It computes a 64-bit digest of the complete state before every operation and requires it unchanged after every refusal, and it checks Proposition 1 directly by rebuilding a fresh table from the live declarations and comparing memory. The program makes 2 732 358 checks and is run under AddressSanitizer and UndefinedBehaviorSanitizer with two compilers.

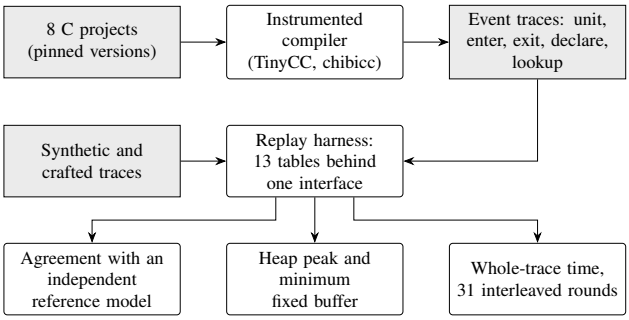


FIGURE 2. Evaluation method. Symbol-table events are logged by instrumented compilers and replayed on every table; an independent reference model decides correctness.

Bounded exhaustive enumeration. A second program enumerates every sequence of nine operations over the alphabet {enter, exit, declare a, b, c, d} from an empty table in eight small configurations chosen so that every kind of refusal occurs, checking after each operation the result, every lookup, the digest after refusals and the structural invariants. This covers all shorter sequences as prefixes: 80 621 568 sequences, 725 594 112 operations, 130 361 632 of them refused, no failure.

Differential fuzzing. A libFuzzer harness lets the input choose the configuration, including the bounded mode and the keyed hash, and up to 4096 operations with names from a colliding pool or arbitrary bytes, with the same checks. The code as it stood before the changes described in Section IX-D ran for 9.8 million executions and the final code for 2.5 million, without a failure.

Mutation check. To know whether these tests can fail, we inject 9 single-line defects, one at a time: a slot not emptied at scope exit, a stale pointer after the buffer moves, a hidden type not restored, the probe limit not enforced, the length of raw names dropped, a refused declaration that still writes, and one defect for each step of the rebuild of Section IV-E. The fuzzer detects all 9 within a second each.

The keyed hash agrees with the reference implementation [24] on 200 000 random inputs. Two static analyzers report nothing on the library.

VIII. EXPERIMENTAL SETUP

A. WORKLOADS

The workloads are sequences of symbol-table events produced by a real compiler rather than by a generator (Fig. 2). We added logging to the Tiny C Compiler [7] at the points where it looks up an identifier, pushes a symbol, and opens or closes a scope, and compiled eight open-source C projects at pinned versions (Table 3). A trace records the start of each translation unit; at that point every table is destroyed and created anew, as a compiler would do. Three Mbed TLS source files that the compiler rejects are absent. A second set of seven traces from an instrumented chibicc [25] is used for the bounded-mode experiment. Nine synthetic traces stress cases that C programs rarely produce, such as a name shadowed in 400 nested scopes.

TABLE 3. The Eight Compiler Traces. "Names" Counts Distinct Identifiers; "Peak" Is the Largest Number of Simultaneously Live Declarations.

Project	Units	Events	Declares	Lookups	Names	Peak	Depth
cJSON	2	20 874	5 247	9 194	1 366	1 067	10
littlefs	2	21 080	3 835	12 230	1 001	732	16
zlib	15	58 058	16 341	27 405	1 041	571	23
Lua	34	188 509	58 619	86 887	3 622	1 365	14
FreeRTOS	9	37 875	10 346	17 417	2 226	1 505	16
Mbed TLS	105	983 008	319 886	438 876	7 622	2 302	14
TinyCC	1	89 135	13 776	54 623	6 704	5 509	27
SQLite	1	268 960	27 072	172 426	7 801	4 245	22

B. TABLES COMPARED

13 tables are driven through one interface.

- STRATA and four variants: raw instead of packed names, a sparse index (at most half full), no shrinking of the buffer, and the keyed hash.
- *The conventional arena table*, our main baseline: bucket heads, a stack of entries (chain link, hash, value, length and depth, unpacked name) and scope marks in one buffer, with the same storage discipline as STRATA: caller-supplied or grown by reallocation, refusal with no state change. It was written for this study as the strongest conventional design we could build, so that a comparison isolates interning, undo entries, packing and open addressing from the mere absence of an allocator.
- One standard-library hash map per scope; an undo log over a standard-library map; the same over a Swiss-table map [26]; an interning Swiss-table map.
- METIS-X [22], the predecessor of this work, a Robin Hood table [13] with one slot per declaration and separate allocations for long names, and two further tables from its repository, all used unmodified at a pinned commit.

The four tables in the third item were written for this study after known designs; they are not the production code of any compiler.

C. METRICS AND PROCEDURE

Correctness is agreement with the reference model on every lookup and every declaration result. *Heap peak* is measured by counting every allocation with the allocator's report of usable size. The *smallest fixed buffer* is found by bisection, at a resolution of 64 bytes, as the smallest buffer with which the whole trace replays without a refusal. *Replay time* is the time to replay a whole trace on a fresh table, including construction and destruction.

Timing on a laptop under a hypervisor drifts, so tables are compared within rounds. In each of 31 measured rounds, after two warm-up rounds, every table replays the trace once, and the starting table rotates from round to round. For two tables we report the median over rounds of the ratio of their times and a 95 percent bootstrap interval from 2000 resamples of

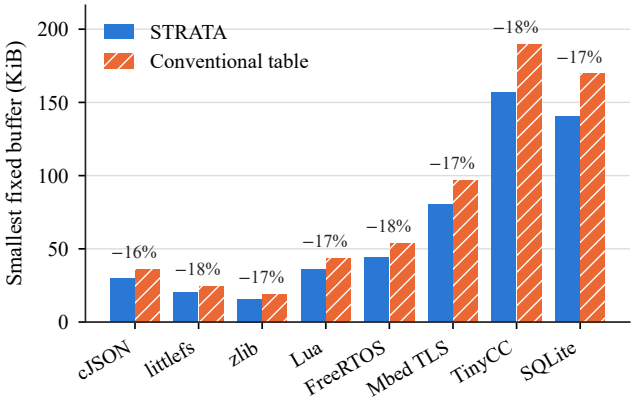


FIGURE 3. Smallest fixed buffer with which each compiler trace replays without a refusal. The label gives the saving of STRATA over the conventional arena table.

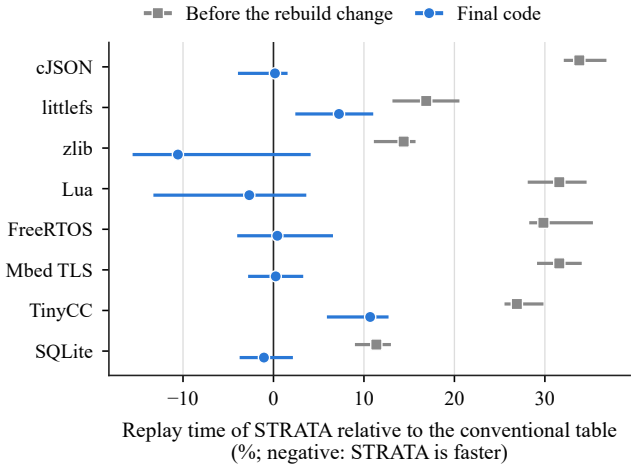


FIGURE 4. Replay time of STRATA relative to the conventional arena table before and after the rebuild change of Section IX-D: median of paired ratios with 95 percent bootstrap interval.

the rounds. We call a difference *established* when the interval excludes zero, and write "n.e." otherwise. Per-operation latency is measured with a timestamp counter around each call and is secondary: values of a few tens of nanoseconds are near the resolution of the method.

D. ENVIRONMENT

All host measurements were taken on one machine: Intel Core Ultra 7 258V, Windows Subsystem for Linux 2 with Ubuntu 22.04 (Linux 6.18.33.2-microsoft-standard-WSL2), g++ 11.4.0 with `-O2`, the benchmark pinned to one core. The measurements reported here are those of run `20261013-143828-paper` of the public artifact [27]. The embedded experiment uses `arm-none-eabi-g++` 10.3.1 with `-Os` and QEMU 6.2 emulating a Cortex-M3 board (`mps2-an385`) restricted to 64 kibibytes (KiB) of memory, without heap and without operating system. Absolute times differ between sessions on this machine by tens of percent; only ratios within a run are compared.

IX. RESULTS

A. CORRECTNESS

All 13 tables agree with the reference model on the eight Tiny C Compiler traces and on the seven chibicc traces: 195 table and trace pairs, no wrong lookup, no wrong declaration result, no refused declaration. On the synthetic traces one table disagrees: METIS-X refuses 149 of 400 declarations when one name is shadowed in 400 nested scopes.

B. MEMORY

Fig. 3 and Table 4 give the memory results. STRATA replays the traces in fixed buffers of 15.8 to 156.5 KiB, which is 27 to 36 bytes per live declaration at the peak. The conventional arena table needs 16.4 to 17.9 percent more on every trace. The saving is nearly constant across projects because it comes from structural differences that every declaration pays for: no stored hash, no chain link, a packed name, and one undo word in place of a full entry when nothing is hidden.

Against tables that allocate, the difference is larger and has a different cause. The heap peak of STRATA in growable mode is 23 to 76 percent (median 58 percent) below that of METIS-X, and STRATA makes 98.7 to 99.9 percent fewer allocator calls. In growable mode the conventional arena table reaches the same heap peak as STRATA on 6 of the 8 traces and a higher one on 2, because both grow one block in steps of one half and the steps mostly hide the finer difference that the fixed-buffer measurement resolves. Most of the gain over the predecessor is therefore due to keeping everything in one allocation-free buffer, not to the compact layout.

C. REPLAY TIME

Table 5 gives the paired comparisons. Against the conventional arena table, STRATA is faster on 0 traces, slower on 2, and on 6 the difference is not established; the point estimates lie between −10.6 and +10.7 percent. The compact table is not faster than the conventional one, and where a difference is established it is at most 10.7 percent. Against METIS-X, STRATA is faster on 8 of 8 traces, by 15.9 to 36.7 percent, and the conventional arena table is faster than METIS-X on 8 of 8, by 14.6 to 34.8 percent. As with memory, most of the improvement over the predecessor is obtained by either single-buffer design.

Per operation (Table 6), the 95th-percentile latency of a declaration is 56 to 66 percent lower with STRATA than with METIS-X and that of a scope exit 71 to 82 percent lower. Lookup latency differs by between −20.3 and +4.9 percent without a consistent direction; values of a few tens of nanoseconds are within the method's resolution of each other.

D. A TIME PENALTY THAT WAS AN ARTIFACT

An earlier version of STRATA was slower than the conventional arena table on every trace, by 11.4 to 33.8 percent (Fig. 4, Table 7), and we initially reported this as the price of the compact design. Per-operation latencies contradicted

TABLE 4. Memory on the Eight Compiler Traces. Smallest Fixed Buffer of the Two Single-Buffer Tables, and Heap Peak in Growable Mode of Six Tables (KiB).

	Smallest fixed buffer				Heap peak, growable mode (KiB)					
Project	STRATA (KiB)	Arena (KiB)	Saving (%)	Bytes per live decl.	STRATA	Arena	METIS-X	Map per scope	Undo log	Interning map
cJSON	30.0	35.9	16.4	28.8	42.9	42.8	102.7	72.0	116.0	140.4
littlefs	20.0	24.2	17.5	28.0	25.8	25.7	106.9	54.1	67.8	141.9
zlib	15.8	18.9	16.6	28.2	17.2	25.7	52.6	44.4	60.3	71.1
Lua	35.9	43.4	17.3	27.0	64.2	96.2	106.8	96.7	127.7	142.3
FreeRTOS	44.0	53.6	17.9	29.9	57.8	57.7	217.3	116.1	147.3	293.1
Mbed TLS	80.0	96.9	17.4	35.6	144.3	144.2	234.4	189.4	277.7	310.9
TinyCC	156.5	189.8	17.5	29.1	324.5	324.4	423.0	424.5	550.8	575.6
SQLite	140.4	169.4	17.2	33.9	194.8	194.7	451.0	333.3	513.5	1125.6

TABLE 5. Replay Time on the Eight Compiler Traces: Median Ratio Over 31 Paired Rounds, in Percent, With 95 Percent Bootstrap Interval. Negative Means That the First Table Is Faster; "n.e." Means Not Established.

Project	STRATA vs. arena table		STRATA vs. METIS-X		Arena table vs. METIS-X		Keyed vs. default hash
cJSON	+0.2 [−3.9, +1.6]	n.e.	−22.8 [−27.5, −19.1]	faster	−21.3 [−23.8, −19.8]	faster	+20.2 [+18.5, +24.6]
littlefs	+7.2 [+2.4, +11.0]	slower	−24.5 [−27.2, −18.1]	faster	−26.5 [−28.9, −22.7]	faster	+24.6 [+18.5, +29.8]
zlib	−10.6 [−15.6, +4.1]	n.e.	−29.9 [−32.8, −27.4]	faster	−20.4 [−26.9, −15.6]	faster	+32.2 [+16.1, +41.4]
Lua	−2.7 [−13.3, +3.6]	n.e.	−30.9 [−35.3, −29.2]	faster	−28.4 [−33.8, −25.2]	faster	+22.7 [+19.1, +35.2]
FreeRTOS	+0.4 [−4.0, +6.6]	n.e.	−33.9 [−39.0, −31.3]	faster	−34.8 [−36.5, −32.9]	faster	+20.3 [+15.2, +22.4]
Mbed TLS	+0.2 [−2.8, +3.3]	n.e.	−36.7 [−39.6, −32.4]	faster	−34.8 [−38.3, −32.2]	faster	+27.1 [+23.0, +34.3]
TinyCC	+10.7 [+5.9, +12.7]	slower	−20.4 [−25.7, −18.7]	faster	−27.5 [−33.7, −22.3]	faster	+20.6 [+13.1, +22.0]
SQLite	−1.1 [−3.7, +2.1]	n.e.	−15.9 [−20.7, −6.3]	faster	−14.6 [−19.5, −10.0]	faster	+22.3 [+18.7, +33.8]

TABLE 6. 95th-Percentile Latency per Operation in Nanoseconds for STRATA (S), the Conventional Arena Table (A) and METIS-X (M).

	Lookup			Declare			Scope exit		
Project	S	A	M	S	A	M	S	A	M
cJSON	49	41	49	62	71	141	13	19	56
littlefs	51	41	49	65	72	187	19	26	90
zlib	44	41	44	56	68	136	16	23	59
Lua	53	89	54	66	130	164	21	56	84
FreeRTOS	52	44	54	68	73	180	15	23	87
Mbed TLS	66	54	69	79	78	202	27	36	109
TinyCC	51	44	51	66	75	188	20	24	76
SQLite	52	53	65	71	86	204	22	28	76

TABLE 7. Replay Time of STRATA Relative to the Conventional Arena Table Before and After the Rebuild Change (Percent, 95 Percent Interval).

Project	Before	After
cJSON	+33.8 [+32.1, +36.8]	+0.2 [−3.9, +1.6]
littlefs	+16.9 [+13.1, +20.6]	+7.2 [+2.4, +11.0]
zlib	+14.4 [+11.1, +15.7]	−10.6 [−15.6, +4.1]
Lua	+31.6 [+28.1, +34.6]	−2.7 [−13.3, +3.6]
FreeRTOS	+29.8 [+28.3, +35.3]	+0.4 [−4.0, +6.6]
Mbed TLS	+31.6 [+29.1, +34.1]	+0.2 [−2.8, +3.3]
TinyCC	+26.9 [+25.5, +29.8]	+10.7 [+5.9, +12.7]
SQLite	+11.4 [+9.0, +13.0]	−1.1 [−3.7, +2.1]

that reading: lookups were level and the whole difference was in declarations. The cause was rebuild work of three kinds. The index was rebuilt on every growth of the buffer, although only the undo stack moves in that case. Each rebuild converted the one-word undo entries from slots to record references and back, hashing every name twice. And a new name walked its probe path twice, once to find that it was absent and once to insert it. Neither name packing nor open addressing was responsible.

After the three changes described in Sections IV and IV-E the penalty is gone on most traces and small on the rest (Section IX-C). The changes alter no result other than time: the correctness counters, the bytes used inside the buffer and the smallest fixed buffers are identical before and after. We report the episode because the first measurement would have supported a plausible but wrong conclusion about the design, and because a baseline strong enough to expose it was what prompted the diagnosis.

E. ABLATION

Table 8 isolates three design choices by switching each off or on. Storing names as raw bytes instead of packing them needs a 5.5 to 9.0 percent larger buffer and is slower on 2 of 8 traces and faster on 0; packing costs no time here because a packed name has fewer words to hash and compare. Keeping the index at most half full instead of three quarters full makes replay faster on 6 of 8 traces (changes between −4.5 and −12.9 percent) and needs a 18 to 27 percent larger buffer, which more than cancels the memory advantage over the conventional table: index load is the lever that trades the saving for speed. Disabling the return of memory to the allocator changes time by between −4.9 and +2.9 percent, faster on 2 traces and slower on 1.

F. SYNTHETIC STRESS TRACES

Table 9 gives the comparison on the 9 synthetic traces, which exaggerate single aspects of a workload. Against the conventional arena table STRATA is faster on 6 and slower on 1. It is faster where names are hidden often, since a hidden declaration costs three words and no name, where scopes are

TABLE 8. Ablation: Change Relative to Default STRATA in Smallest Fixed Buffer and in Replay Time (Percent; Verdict From the 95 Percent Interval).

	Raw names		Sparse index		No shrinking
Project	Buffer	Time	Buffer	Time	Time
cJSON	+5.8	−0.2 (n.e.)	+26.7	−12.1 (faster)	−2.8 (faster)
littlefs	+7.5	+4.4 (n.e.)	+20.0	−12.9 (faster)	+0.7 (n.e.)
zlib	+6.0	+3.1 (n.e.)	+25.4	−9.0 (faster)	−4.9 (faster)
Lua	+6.1	+7.1 (slower)	+22.3	−11.8 (faster)	+1.4 (n.e.)
FreeRTOS	+8.5	+3.0 (n.e.)	+18.2	−9.3 (faster)	−4.4 (n.e.)
Mbed TLS	+9.9	−1.6 (n.e.)	+20.0	−4.8 (n.e.)	+2.7 (slower)
TinyCC	+7.5	+9.9 (slower)	+20.4	−9.2 (faster)	+1.2 (n.e.)
SQLite	+8.9	−0.2 (n.e.)	+22.8	−4.5 (n.e.)	+2.9 (n.e.)

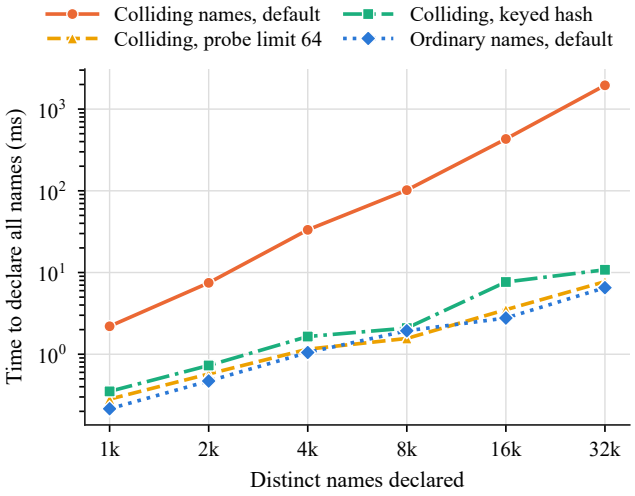


FIGURE 5. Time to declare *N* distinct names of 150 characters. Colliding names are those of Section VI-D; ordinary names are random identifiers of the same length. Both axes are logarithmic.

entered and left at a high rate or are very large, and with short names. It is slower on the trace dominated by lookups, and the difference is not established on the traces with long names and with 50 000 names in one scope.

G. HASH FLOODING AND ITS DEFENSES

We declare and then look up the 32 768 colliding identifiers of Section VI-D and, for comparison, as many random identifiers of the same length (Fig. 5, Table 10). In the default configuration the colliding names take 1954 ms to declare against 6.5 ms for ordinary names, a factor of 300, and 1684 ms to look up. The time grows with exponent 1.96 in the number of names: every new name scans the cluster formed by all earlier ones.

With the keyed hash the factor is 1.2: names built against the unkeyed mixer are ordinary names under a key their author does not know. With the unkeyed hash and a probe limit of 64, time grows linearly, because no call examines more than 64 slots; the price is that 32 704 of the 32 768 colliding names are refused. The two defenses address different halves of the problem: the limit bounds the work of a call whatever the hash does, and the key keeps an adversary from causing refusals.

The keyed hash is not free. On the compiler traces it increases replay time by 20.2 to 32.2 percent (Table 5).

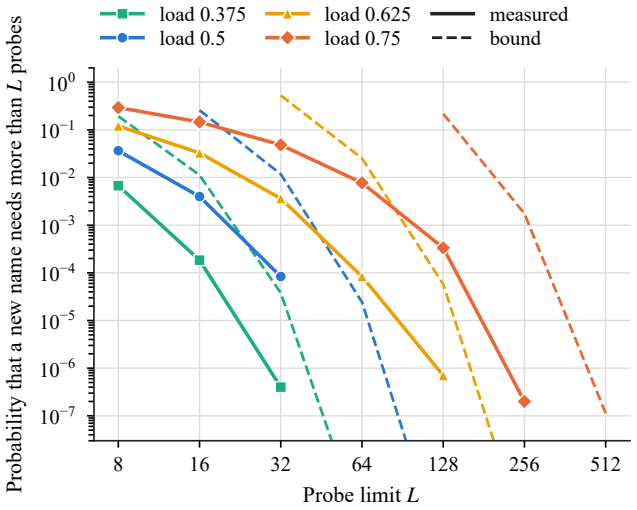


FIGURE 6. Probability that the insertion of a fresh name examines more than *L* index slots, for four loads of an index of 65 536 slots under the keyed hash: measured on 10 000 000 insertions per load, and the bound (2) where it is below one.

H. THE PROBE LIMIT IN SIMULATION AND ON REAL TRACES

To validate Proposition 2 we fill an index by linear probing with random identifiers of 6 to 24 characters, taking home slots from the table's own hash of each name, and record how many slots the insertion of a fresh name would examine. We do this for three index sizes, four loads and both hashes, with 500 tables and 10 000 000 fresh names per configuration: 24 configurations and 240 000 000 samples (Fig. 6, Table 11).

Three observations follow. The measured mean number of probes agrees with Knuth's value for insertion-only linear probing at every load and for both hashes, which is Proposition 1 and the uniformity assumption seen in data. The measured probability never exceeds the bound in any of the 168 combinations of configuration and limit; the bound is loose by about two orders of magnitude where both are nonzero, as a union-and-Chernoff bound will be. And at the load limit of $3/4$, 0.77 percent of fresh names need more than 64 probes.

The last observation corrects a conclusion that the compiler traces alone suggest. Replaying the 15 compiler traces (719 833 declarations) in bounded mode, with the index sized for each trace's peak, a limit of 64 refuses no declaration with either hash, and the longest probe sequence is 47 (Table 12); every lookup agrees with the reference model, the index is never rebuilt, and the peak memory equals that of the growing index. From this one might adopt 64 as a safe setting. It is not: those tables are small and spend most of their time well below the load limit. At full load a limit of 256 was exceeded with probability 2.0×10^{-7} in simulation and a limit of 512 not at all in 10 000 000 samples, for which the bound gives at most 1.1×10^{-7}. We therefore recommend a limit of 512 with the default index, or 128 with an index kept at most half full

TABLE 9. Synthetic Stress Traces: Replay Time of STRATA Relative to the Conventional Arena Table and to METIS-X (Percent, With Verdict), and Whether METIS-X Agrees With the Reference Model.

Trace	Declarations	Maximum depth	STRATA vs. arena table	STRATA vs. METIS-X	METIS-X result
`big_scope_exit`	200 100	1	−39.0 (faster)	−53.6 (faster)	agrees
`deep_nesting`	89 949	200	−12.1 (faster)	−32.1 (faster)	agrees
`deep_shadow_400`	400	400	−51.3 (faster)	−96.4 (faster)	149 refused
`dense_50k`	50 000	0	+0.7 (n.e.)	−26.8 (faster)	agrees
`long_ids`	89 899	10	+6.9 (n.e.)	−38.9 (faster)	agrees
`lookup_heavy`	15 097	10	+12.5 (slower)	−0.1 (n.e.)	agrees
`scope_churn`	150 050	1	−24.7 (faster)	−26.3 (faster)	agrees
`shadow_heavy`	134 903	40	−21.2 (faster)	−54.3 (faster)	agrees
`short_ids`	89 646	10	−16.3 (faster)	−36.0 (faster)	agrees

TABLE 10. Declaring 32 768 Names of 150 Characters in Four Configurations: Time in Milliseconds, Their Ratio, Refused Names and Longest Probe Sequence for the Colliding Input.

Configuration	Ordinary	Colliding	Ratio	Refused	Longest
Default	6.52	1953.86	299.7	0	32 768
Keyed hash	9.33	10.80	1.2	0	106
Limit 64	3.11	7.72	2.5	32 704	64
Limit 64, keyed	5.22	5.56	1.1	0	26

(bound 1.0×10^{-10}), and regard 64 as an engineering choice for callers who accept occasional refusals. On the compiler traces the recommended limit behaves as any limit of 64 or more does: nothing is refused (Table 12).

I. WHEN DOES THE SMALLER BUFFER MATTER?

A saving of one sixth decides whether a workload fits only when the budget lies between the needs of the two tables. Table 13 counts, for the 169 translation units of the eight projects, how many fit at a given budget. The units that only STRATA can hold are few at every budget, at most 23 at 16 KiB. The benefit of the compact layout is real but narrow; where it applies, it is the difference between compiling a unit and refusing it.

A sweep over the shadowing factor (live declarations per distinct name) and the name length confirms the layout model (1): the saving over the conventional table grows with shadowing, since a hidden declaration costs three words in STRATA and a full entry with the name in the conventional table, and reaches 65 percent at the highest factor we tested (Table 14). Replay time relative to the conventional table follows the name length rather than the shadowing factor: STRATA is faster with 6-character names at every factor and slower with 24-character names at most factors.

J. INSIDE A COMPILER

Replay measures a table in isolation. To see what the differences amount to in a whole compiler, we replaced the per-scope hash maps that chibicc [25] uses for variable scopes by STRATA and, separately, by the conventional arena table, and compiled seven of the projects with all three builds. The unmodified compiler accepts 158 of their 170 translation units; on those, the assembly produced with STRATA is byte-for-byte identical to the original on 158 units and with the arena table on 158.

Table 15 gives whole-project compile time over 21 interleaved rounds. With STRATA the change lies between −3.7 and +2.2 percent, established slower on 1 project and faster on 0; with the arena table between −3.1 and +4.3 percent, faster on 3 and slower on 0. Peak resident memory per unit changes by a median of 0.18 percent. In a hosted compiler, then, neither single-buffer table matters: symbol-table work is a small part of the compile, and the differences that the replay experiments resolve disappear in the noise of the whole. Both builds also keep file-scope declarations in a second table to answer the compiler's global lookups, which is work the original does not do. The experiment establishes that the tables are correct in a real resolution path; it does not show a benefit, and none should be claimed for compilers that are not memory-constrained.

K. AN EMULATED MICROCONTROLLER

To check that the fixed-buffer mode needs neither a heap nor a 64-bit machine, we built a bare-metal image for a Cortex-M3 and ran it under QEMU with 64 KiB of memory. Allocation functions are stubbed to fail. Five of the eight traces fit that memory; all 5 replay with a checksum over every result identical to the host's, using 492 bytes of stack, and the table's code occupies 2 715 bytes. The same 5 traces pass in bounded mode with the keyed hash and a probe limit of 512, with no refusal. The emulator provides no timing, and no physical device was used.

X. DISCUSSION

What the compact design buys. Against a conventional chained table that obeys the same storage discipline, STRATA offers a buffer about one sixth smaller at a speed that is indistinguishable on most traces and up to 10.7 percent lower on the others. That is a modest result, and it is narrower than the improvement over the multi-allocation predecessor suggests: most of that improvement belongs to the storage discipline, which the conventional design shares. For a compiler on a workstation neither table matters. For a front end with tens of kilobytes, one sixth of the symbol table is memory that can go elsewhere, and Section IX-I shows how often it changes the outcome.

What the storage discipline buys. One buffer and write-after-reserve give a refusal that is simple to state and, as Section VII shows, simple to test exhaustively at small scale. We

TABLE 11. Insertion Probe Lengths Under the Keyed Hash, Index of 65 536 Slots, 10 000 000 Samples per Row: Mean Against Knuth's Value, Longest Observed, and Measured Probability of Exceeding a Limit Against the Bound (2) ("n/a" Where the Bound Exceeds One).

Load	Mean	Knuth	Longest	> 64: measured / bound	> 128	> 256	> 512
0.375	1.78	1.78	44	$0 / 4.3\times10^{-10}$	$0 / 5.5\times10^{-20}$	$0 / 9.2\times10^{-40}$	$0 / 2.5\times10^{-79}$
0.500	2.50	2.50	63	$0 / 2.4\times10^{-5}$	$0 / 1.0\times10^{-10}$	$0 / 1.9\times10^{-21}$	$0 / 6.4\times10^{-43}$
0.625	4.05	4.06	144	$8.4\times10^{-5} / 2.5\times10^{-2}$	$7.0\times10^{-7} / 5.8\times10^{-5}$	$0 / 3.0\times10^{-10}$	$0 / 8.3\times10^{-21}$
0.750	8.50	8.50	258	7.7×10^{-3} / n/a	$3.3\times10^{-4} / 2.2\times10^{-1}$	$2.0\times10^{-7} / 1.7\times10^{-3}$	$0 / 1.1\times10^{-7}$

TABLE 12. Bounded Mode on 15 Compiler Traces (719 833 Declarations): Declarations Refused by the Probe Limit and Number of Traces Affected, for the Default and the Keyed Hash.

	Default hash		Keyed hash	
Limit	Refused	Traces	Refused	Traces
4	15 335	15	15 589	15
8	3 543	15	3 507	15
16	561	12	571	11
32	51	5	51	4
64	0	0	0	0
128	0	0	0	0
256	0	0	0	0
512	0	0	0	0

TABLE 13. Translation Units (of 169) That Fit a Fixed Budget: in Both Single-Buffer Tables, Only in STRATA, or in Neither.

Budget (KiB)	Both	Only STRATA	Neither
8	1	1	167
16	28	23	118
32	100	7	62
64	146	1	22
128	167	0	2

TABLE 14. Sweep Over the Shadowing Factor σ and the Name Length: Saving of STRATA in Smallest Fixed Buffer (Percent) and Replay Time Relative to the Conventional Arena Table (Percent).

	6 characters		12 characters		24 characters	
σ	Save	Time	Save	Time	Save	Time
1.0	0	−30.7 (faster)	0	+0.6 (n.e.)	9	+6.1 (n.e.)
1.5	24	−36.7 (faster)	25	+5.5 (n.e.)	33	+5.8 (slower)
2.0	29	−35.2 (faster)	31	+8.1 (slower)	41	+12.2 (slower)
4.0	43	−47.5 (faster)	47	−4.4 (n.e.)	57	+16.0 (slower)
8.0	50	−39.9 (faster)	55	+2.8 (n.e.)	65	+20.9 (slower)

consider this the more transferable result. It applies equally to the conventional table, which also passes the reference comparison on every trace.

Why last-in-first-out removal matters beyond speed. The removal rule of Section IV-C was adopted to avoid tombstones. Its more useful consequence is Proposition 1: because the table is always in the state of an insertion-only table, one can reason about it with the simplest analysis of linear probing. Without that property the bound of Section VI-B would have had to account for the history of deletions.

Why measurement alone was not enough. Twice in this work a measurement supported a conclusion that further analysis overturned: the time penalty of Section IX-D, and the probe limit of 64 in Section IX-H. In the first case a strong baseline and per-operation data exposed the artifact; in the second a bound showed that the traces were unrepresentative of the worst legitimate case. We regard both as arguments for pairing replay experiments with a baseline that shares the design's constraints and with an analysis of the quantity being tuned.

Where the design is the wrong choice. If memory is not constrained and the input is trusted, this study gives no reason to prefer a single-buffer table: the allocator-backed tables are simpler to adopt, and their larger memory (Table 4) does not matter there. If the input is untrusted, the keyed hash costs 20.2 to 32.2 percent of replay time, and a caller must weigh that against the threat.

XI. LIMITATIONS AND THREATS TO VALIDITY

Mostly replay, not compilation. The traces record what a compiler's own tables were asked to do; replaying them measures a table, not a compiler built on it. The one experiment inside a compiler (Section IX-J) replaces variable scopes only, on a hosted machine, and shows no end-to-end benefit. No compiler was run on the table under an actual memory budget.

One machine, no hardware. All timing comes from one laptop under a hypervisor. The microcontroller is emulated and yields no timing. The interquartile range of raw replay times is 16 percent of the median for the typical table and trace, which the paired design mitigates but does not remove. The verdicts on individual traces are not stable across runs: in an earlier run of the same code STRATA was faster than the conventional arena table on two traces and slower on three, with point estimates between -10.7 and $+7.0$ percent. What both runs support is that the two tables are within about ten percent of each other, not a ranking trace by trace.

Workloads. The traces are C only, from two small compilers whose lookup behavior differs from that of production compilers, and they depend on the host's system headers. Names containing characters outside the identifier alphabet, as in mangled C++ names, take the unpacked path, which the traces exercise little.

Baselines. The conventional arena table and four further baselines were written by the authors of the table under study. We wrote the arena table to be strong, and it is the baseline that changed our conclusions; nevertheless it is not an independent implementation. LLVM's and Clang's own containers were not measured.

The bound and the keyed hash. Proposition 2 assumes an ideal hash, and its use against an adversary assumes that Half-SipHash with one and three rounds and a 64-bit key behaves as one, which its designers and the Linux kernel documentation consider adequate for hash-table flooding only. We did not analyze side channels. The simulation uses random identifiers, not program text. On a big-endian target the keyed

TABLE 15. End to End in chibicc: Whole-Project Compile Time With Variable Scopes on STRATA or on the Conventional Arena Table, Relative to the Original Compiler (Percent, 95 Percent Interval; Negative Is Faster).

Project	Units compiled	Original compiler (s)	STRATA		Arena table	
cJSON	2	0.11	−0.7 [−5.9, +8.8]	n.e.	+4.3 [−2.6, +10.7]	n.e.
littlefs	1	0.02	−3.7 [−5.8, +2.6]	n.e.	−3.1 [−6.4, −0.7]	faster
zlib	12	0.30	+2.2 [−3.0, +5.7]	n.e.	+0.5 [−3.9, +4.9]	n.e.
Lua	34	1.09	−0.7 [−3.5, +0.5]	n.e.	−1.8 [−2.7, −0.3]	faster
FreeRTOS	7	0.65	+1.8 [−4.1, +5.5]	n.e.	−0.5 [−7.0, +7.8]	n.e.
Mbed TLS	101	5.58	+1.4 [+0.2, +2.4]	slower	+1.0 [−0.7, +1.5]	n.e.
SQLite	1	1.00	+0.0 [−3.3, +2.8]	n.e.	−2.1 [−3.6, −0.2]	faster

hash would compute different, though consistent, values; this was not tested.

The attack. The collision construction is a known technique; our part is its restriction to packed identifier words and the measurement.

Verification. Fuzzing and bounded enumeration are evidence, not proof. The enumeration covers sequences of nine operations over four names. The arguments for the two propositions are informal and were not machine-checked.

Design limits. Names of at most 255 bytes, 16 383 nested scopes, a buffer of at most 64 MiB, one name space per table, single-threaded use. In the default mode a call that triggers a rebuild takes time proportional to the number of visible names.

Literature. The search for related work was limited in scope and several sources were consulted at the level of abstracts or documentation. A closer precedent for a budgeted scoped table may exist.

XII. CONCLUSION

A scoped symbol table can be confined to one fixed buffer, refuse cleanly when the buffer is full, and bound the work of each call on hostile input, at a cost that we have measured rather than assumed. On events logged by a C compiler for eight projects, the compact design studied here needs a buffer about one sixth smaller than a conventional single-buffer table and is within about ten percent of it in time, with no established difference on most traces; both are well ahead of a multi-allocation predecessor, and that larger gain belongs to the storage discipline rather than to the compact layout. Because declarations are removed in the reverse order of their creation, the table is always in the state of an insertion-only table, which makes a simple bound on the probe limit valid; the bound and a simulation agree that a limit suggested by real traces would have been several times too small. Identifiers that collide for every seed of the default hash slow declarations by more than two orders of magnitude, and a keyed hash removes the effect at a cost of roughly a quarter of replay time.

The most useful next steps are those that remove the limitations of Section XI: a compiler that actually runs on the table on a physical microcontroller, a machine-checked proof of the two propositions, workloads from languages other than C, and a sharper bound than the one obtained here by a union and a Chernoff argument.

DATA AND CODE AVAILABILITY

The implementation, the tests, the compiler patches, the traces, the raw measurements of every run cited here, and the scripts that regenerate every number, table and figure of this article from those measurements are public [27].

ACKNOWLEDGMENT

The authors thank Bhuvaneswari M. for guiding the project from which this work grew. **[AUTHOR: confirm the wording and the guide's affiliation.]**

In accordance with the journal's policy on generated text: the implementation, the experiments, the analysis and the text of this manuscript were produced with the assistance of an artificial-intelligence coding assistant (Claude, Anthropic) working at the direction of the first author; this applies to every section. **[AUTHOR: the authors must review this statement, verify every result themselves, and confirm that they accept responsibility for the entire content.]**

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.

[2] S. L. Graham, W. N. Joy, and O. Roubine, "Hashed symbol tables for languages with explicit scope control," in *Proc. SIGPLAN Symposium on Compiler Construction*, 1979, pp. 50–57.

[3] A. W. Appel, *Modern Compiler Implementation in C*. Cambridge, U.K.: Cambridge University Press, 1998.

[4] S. A. Crosby and D. S. Wallach, "Denial of service via algorithmic complexity attacks," in *Proc. 12th USENIX Security Symposium*, 2003, [Online]. Available: https://www.usenix.org/conference/12th-usenix-security-symposium/denial-service-algorithmic-complexity-attacks.

[5] J.-P. Aumasson and D. J. Bernstein, "SipHash: A fast short-input PRF," in *Progress in Cryptology – INDOCRYPT 2012*, ser. Lecture Notes in Computer Science, vol. 7668. Springer, 2012, pp. 489–508.

[6] LLVM Project, "ScopedHashTable.h, LLVM ADT library," [Online]. Available: https://llvm.org/doxygen/ScopedHashTable_8h_source.html. Accessed: Oct. 13, 2026.

[7] F. Bellard and TinyCC contributors, "Tiny C Compiler, commit 43c7708," [Online]. Available: https://repo.or.cz/tinycc.git. Accessed: Oct. 11, 2026.

[8] Rust Project Developers, "rustc-hash: A speedy, non-cryptographic hashing algorithm used by rustc," [Online]. Available: https://github.com/rust-lang/rustc-hash. Accessed: Oct. 13, 2026.

[9] D. R. Hanson, "Fast allocation and deallocation of memory based on object lifetimes," *Software: Practice and Experience*, vol. 20, no. 1, pp. 5–12, 1990.

[10] MicroPython Project, "MicroPython string interning," [Online]. Available: https://docs.micropython.org/en/latest/develop/qstr.html. Accessed: Oct. 13, 2026.

[11] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998.

[12] M. A. Bender, B. C. Kuszmaul, and W. Kuszmaul, "Linear probing revisited: Tombstones mark the demise of primary clustering," in *Proc. IEEE*

62nd Annual Symposium on Foundations of Computer Science (FOCS), 2021, pp. 1171–1182.

[13] P. Celis, P.-Å. Larson, and J. I. Munro, "Robin Hood hashing," in *Proc. 26th Annual Symposium on Foundations of Computer Science (SFCS)*, 1985, pp. 281–288.

[14] M. Naor and V. Teague, "Anti-persistence: History independent data structures," in *Proc. 33rd Annual ACM Symposium on Theory of Computing (STOC)*, 2001, pp. 492–501.

[15] G. E. Blelloch and D. Golovin, "Strongly history-independent hashing with applications," in *Proc. 48th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 2007, pp. 272–282.

[16] A. Pagh, R. Pagh, and M. Ružić, "Linear probing with constant independence," in *Proc. 39th Annual ACM Symposium on Theory of Computing (STOC)*, 2007, pp. 318–327.

[17] M. Thorup, "Linear probing with 5-independent hashing," arXiv:1509.04549, 2015, [Online]. Available: https://arxiv.org/abs/1509.04549.

[18] R. Pagh and F. F. Rodler, "Cuckoo hashing," *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.

[19] M. Herlihy, N. Shavit, and M. Tzafrir, "Hopscotch hashing," in *Distributed Computing*, ser. Lecture Notes in Computer Science. Springer, 2008, pp. 350–364.

[20] J.-P. Aumasson, D. J. Bernstein, and M. Boßlet, "Hash-flooding DoS reloaded: Attacks and defenses," Presentation slides, 2012, [Online]. Available: https://www.aumasson.jp/siphash/siphashdos_appsec12_slides.pdf.

[21] J. A. Donenfeld, "SipHash – a short input PRF," Linux kernel documentation, [Online]. Available: https://docs.kernel.org/security/siphash.html. Accessed: Oct. 13, 2026.

[22] A. Jain and S. Singh, "METIS: Symbol-table research repository, commit 7b3dacf," 2026, [Online]. Available: https://github.com/adhyan-jain/Metis. Accessed: Oct. 11, 2026.

[23] A. Appleby, "MurmurHash3, in SMHasher," [Online]. Available: https://github.com/aappleby/smhasher. Accessed: Oct. 13, 2026.

[24] J.-P. Aumasson, "SipHash and HalfSipHash reference implementation," [Online]. Available: https://github.com/veorq/SipHash. Accessed: Oct. 13, 2026.

[25] R. Ueyama, "chibicc: A small C compiler, commit 90d1f7f," [Online]. Available: https://github.com/rui314/chibicc. Accessed: Oct. 11, 2026.

[26] G. Popovitch, "The parallel hashmap, version 1.4.1," [Online]. Available: https://github.com/greg7mdp/parallel-hashmap. Accessed: Oct. 13, 2026.

[27] S. Singh, A. Jain, A. Venkatataramana, and Arjuman, "STRATA: Source code, traces, and measurements," 2026, [Online]. Available: https://github.com/shubhisingh1510/cdesproject.

SHUBHI SINGH **[AUTHOR: biography: degrees and institutions, current position, research interests. Add a photograph of 1 in by 1.25 in and switch to the IEEEbiography environment.]**

ADHYAN JAIN **[AUTHOR: biography and photograph.]**

AKSHITH VENKATATARAMANA **[AUTHOR: biography and photograph; confirm the spelling of the name.]**

ARJUMAN **[AUTHOR: biography and photograph; confirm the full name.]**